2026/2027

ESTUDIO TEMA 1

DWES



JUAN MIGUEL DOMÍNGUEZ PERDIGÓN
IES LOS SAUCES – DAW2

TEMA 1 DESARROLLO WEB EN ENTORNO SERVIDOR

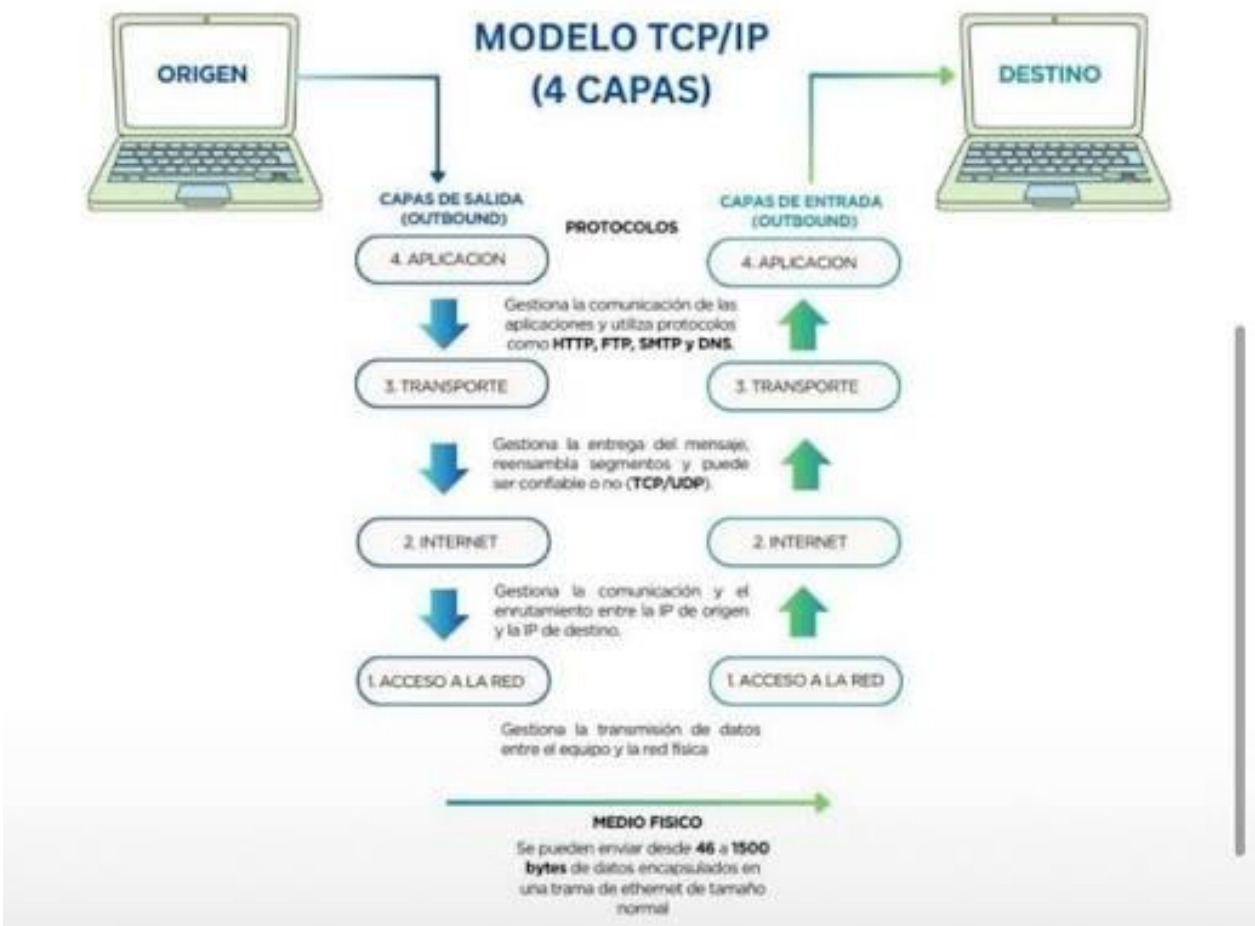
EJERCICIOS 2
1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS. 2
2. Modelo de comunicaciones cliente – servidor 6
3. Métodos de petición HTTP/HTTPS más utilizados 7
4. EL concepto de URI/URL/URN 8
5. Modelo Multicapa 9
6. Front-end / Back-end 11
7. Tipos de web 12
8. Componentes de una aplicación web 14
9. Programas en cliente y servidor 15
10. Lenguajes lado servidor 17
11. Plataforma XAMPP 18
12. Uso de JVM y JDK 19
13. IDE más utilizados 20
14. Servidores HTTP/HTTPS 21
15. Apache HTTP vs Apache Tomcat 22
16. Navegadores 23

EJERCICIOS

1. Protocolos de comunicaciones: IP, TCP, HTTP, HTTPS.

IP, TCP HTTP, HTTPS son protocolos que forman parte del modelo TCP/IP. El modelo TCP/IP es un conjunto de protocolos y una arquitectura de red que permite la comunicación entre dispositivos conectados a una red.

Capa	Función Principal	Ejemplos
Aplicación	Proporciona servicios de red a las aplicaciones, formatea y codifica la página web	HTTP, HTTPS, DNS
Transporte	Comunicación entre aplicaciones y control de la transmisión, segmenta los datos	TCP
Internet	Direccionamiento y encaminamiento de paquetes	IP
Acceso a red	Comunicación con la red física y enlace de datos	Ethernet, Wi-Fi

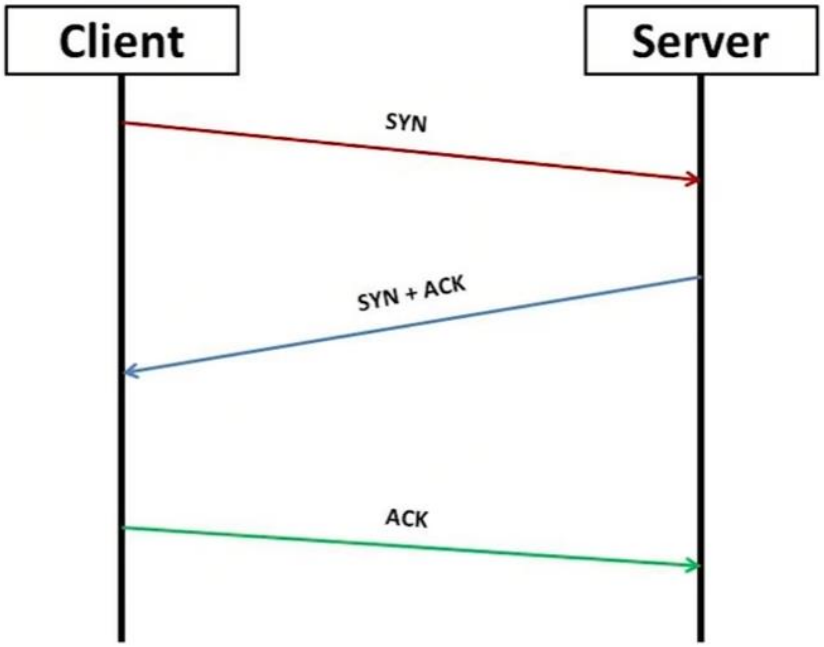




HTTP (Hypertext Transfer Protocol) es un protocolo que gestiona la comunicación de las aplicaciones. Es utilizado principalmente para transferir recursos web entre clientes y servidores. Por ejemplo, un cliente le hace una petición al servidor buscando la página www.elpais.es en su navegador, el servidor busca si existe y responde con el código de estado 200 y la página. Pero si la página no existe el servidor responderá con el código de estado 404.

HTTPS (HyperText Transfer Protocol Secure) es la utilización de HTTP sobre una conexión protegida mediante TLS (Transport Layer Security), proporcionando una protección criptográfica. Su objetivo principal es proteger la comunicación entre el cliente y el servidor.

TCP (Transmission Control Protocol) es un protocolo de la capa de transporte diseñado para proporcionar una comunicación fiable, ordenada y orientada a conexión entre dos aplicaciones, a diferencia de UDP. Una vez los datos hayan sido segmentados, el protocolo le da un encabezado a los segmentos, de cuya encabezado destaca el puerto de origen y el puerto de destino. Este protocolo asegura que los datos lleguen al receptor, primero estableciendo una conexión denominada como three-way handshake o negociación de tres pasos. De forma simplificada: 1. El cliente envía SYN; 2. El servidor responde SYN-ACK; 3. El cliente responde ACK. Una vez completado este proceso, comienza la transferencia de datos. El protocolo también enumera sus paquetes al envíralos, una vez recibidos vuelven a su orden original gracias a la enumeración.



IP (Internet Protocol) es el protocolo responsable del direccionamiento y encaminamiento de los paquetes dentro de una red basada en Internet. Su función principal es conseguir que los paquetes puedan viajar desde un origen hasta un destino. IP funciona mediante un sistema de direcciones IP, que identifica las interfaces de red. El protocolo empaqueta los segmentos y los encabeza con la dirección IP de origen y de destino, definida como información de direccionamiento lógico.



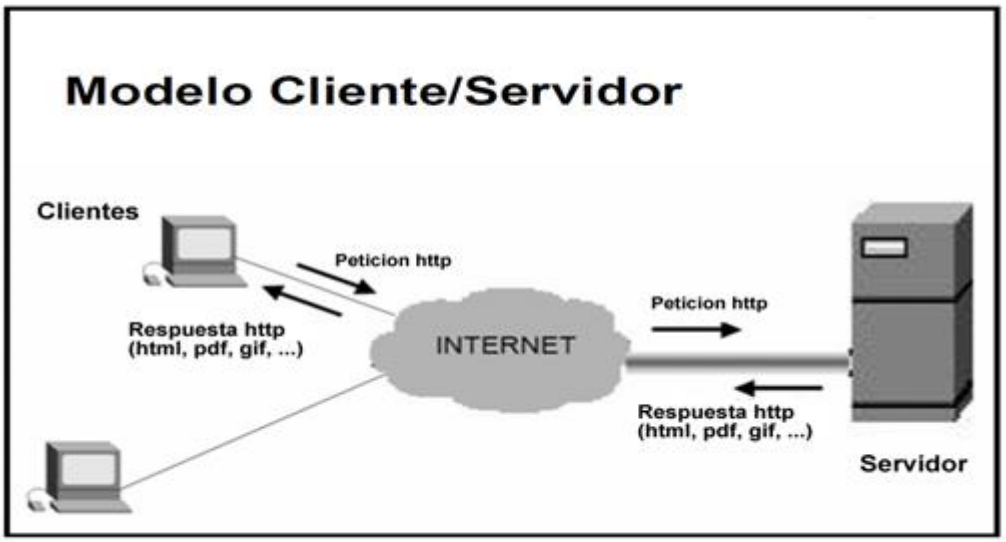
2. Modelo de comunicaciones cliente – servidor

Es una arquitectura donde un dispositivo (cliente) solicita recursos o servicios a otro (servidor), que los procesa y devuelve una respuesta. En las aplicaciones web, el navegador actúa como cliente haciendo peticiones HTTP a un servidor web que aloja la aplicación.

El modelo cliente-servidor es un modelo de diseño de software distribuido en la que dos tipos de dispositivos o programas tienen funciones diferentes. El "cliente" es el dispositivo o aplicación que solicita un servicio. Por ejemplo, un navegador web. Mientras que el "servidor" es el equipo que recibe las solicitudes, las procesa y devuelve una respuesta. La comunicación entre el cliente y servidor se realiza mediante protocolos de red como TCP/IP y HTTP/HTTPS.

Las aplicaciones web utilizan este modelo constantemente. El navegador o cliente se comunica con un servidor mediante HTTP/HTTPS para solicitar páginas, imágenes o datos. Un ejemplo, un navegador el cliente busca www.elpais.es. Al buscar esto en el navegador el dispositivo manda una señal al servidor. Una vez lo recibida servidor, él responde con los datos necesarios para construir la página al navegador, construyendo la página en el navegador.

Navegador → Solicitud → Servidor → Respuesta → Navegador

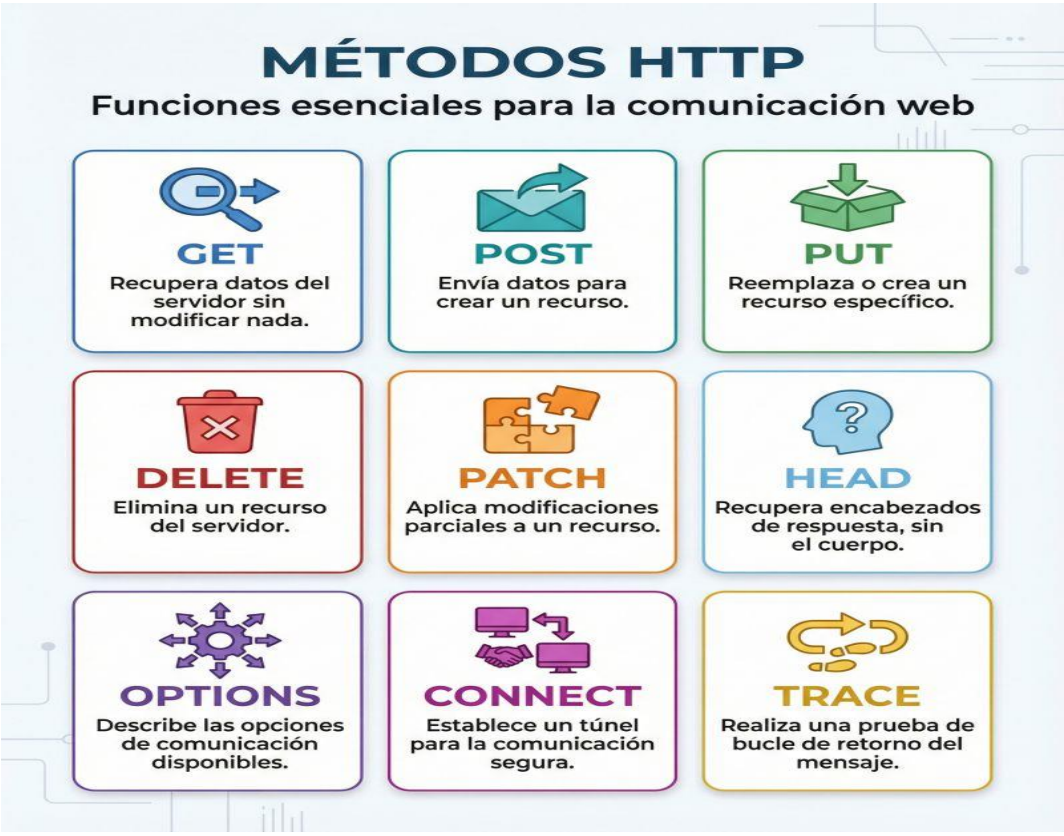


Las ventajas de utilizar este modelo son que permite centralizar la información y servicios en un solo servidor. Facilita la gestión y actualización de las aplicaciones. También permite que muchos clientes accedan al servidor al mismo tiempo. El protocolo HTTPS permite proteger la comunicación mediante cifrado.

3. Métodos de petición HTTP/HTTPS más utilizados

Los métodos HTTP indican la acción que el cliente desea realizar sobre un recurso, siendo GET para solicitar datos y POST para enviar información al servidor. Otros métodos comunes incluyen PUT o PATCH para actualizar información existente y DELETE para borrarla.

Son métodos de petición HTTP/HTTPS comandos que se le envían al servidor web mediante el protocolo HTTP/HTTPS.
GET: solicita y recupera información o recursos específicos de un servidor sin alterar su estado. Ejemplo: archivos HTML, imágenes, y datos de una API
POST: se utiliza para enviar datos al servidor, común al rellenar formularios, subir archivos o crear nuevos registros y recursos.
PUT: reemplaza un recurso existente por completo o crea uno nuevo si no existe
DELETE: elimina un recurso específico del servidor
PATCH: Aplica modificaciones parciales a un recurso sin tener que enviarlo completo.
HEAD: solicita las mismas cabeceras que un método GET, pero sin devolver el cuerpo de la respuesta.
TRACE: este método solicita al servidor que envié de vuelta en un mensaje
OPTIONS: Describe las opciones de comunicación disponibles para un recurso especifico.
CONNECT: se utiliza para saber si se tiene acceso a un host.



4. EL concepto de URI/URL/URN

Un URI (Uniform Resource Identifier) es una cadena que identifica un recurso en la red, abarcando tanto su nombre (Uniform Resource Name) como su ubicación exacta (Uniform Resource Locator). En la web, la URL es vital porque indica al navegador el protocolo HTTP/HTTPS, el dominio y la ruta específica del recurso a solicitar.

URI un sistema que ha sido diseñado para nombrar y localizar los recursos de la web de manera unívoca.

La estructura genérica de un URI se describe a continuación:

[protocolo][nombre_del_equipo][ruta_directorios][puerto][archivo]

Protocolo: También llamado esquema URL, este componente especifica el protocolo que se requiere utilizar para acceder al documento o recurso. Lista de portocolos

Nombre del equipo: (Habitualmente un nombre de dominio o una dirección IP) donde el contenido está alojado.

Ruta de directorios: Una secuencia de directorios separados por barras que define la ruta a seguir en la computadora para llegar al archivo.

Archivo: El nombre del archivo que contiene al recurso.

Parametros al final dominio no ruta=index

Ejemplo de un URI con todas sus partes etiquetadas:



No es requerida la utilización de todas estas componentes para conformar un URI válido. Muchos URIs pueden tener sólo algunas de estas partes presentes (o ninguna) y aún ser válidas.

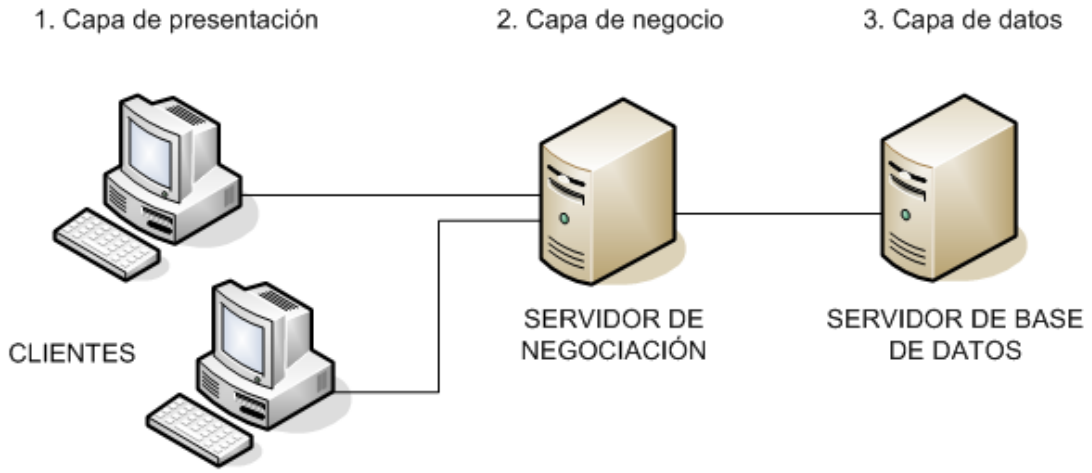
Algunos ejemplos de URIs válidos:

- Nombre de la computadora `195.156.123.1` Directorios `/backup/` (sin protocolo; sin archivo)
- Protocolo `http://` Nombre de la computadora `yeahyeah.com` `/` (sin directorios; sin archivo)
- Directorios `/john/trabajo/` Archivo `reporte.odt` (sin protocolo; sin nombre de la computadora)
- Archivo `index.html` (sin protocolo; sin nombre de computadora; sin directorios)
- `/` (sin protocolo; sin nombre de computadora; sin directorios; sin archivo)

5. Modelo Multicapa

Este modelo divide la aplicación en niveles lógicos separados (generalmente presentación, lógica de negocio y datos) para facilitar el mantenimiento y la escalabilidad. Cada capa se comunica exclusivamente con sus capas adyacentes mediante interfaces bien definidas, procesando la información antes de delegarla.

El modelo de desarrollo de aplicaciones multicapa divide el software en capas: Capa de Presentación (interfaz de usuario), Capa de Lógica de negocio (procesamiento de reglas y servidor web) y Capa de Datos (Bases de datos). Facilita el mantenimiento, seguridad y escalabilidad.



- Capa de presentación:

Presenta el sistema al usuario, le comunica la información y captura la información del usuario en un proceso. Esta capa se comunica únicamente con la capa de negocio. Sus características principales son:

- Recoge la información del usuario y la envía al servidor (cliente).
- Manda información a la capa de proceso para su procesado.
- Recibe los resultados de la capa de proceso.
- Generan la presentación.
- Visualizan la presentación al usuario (cliente).

- Capa de negocio:

Es donde residen los programas que se ejecutan, reciben las peticiones del usuario y se envían las respuestas tras el proceso. Esta capa se comunica con la capa de presentación, para recibir las solicitudes y presentar los resultados. Si la aplicación se comunica con otros sistemas que actúan en conjunto, lo hace mediante esta capa. También se comunica con la capa de datos para obtener información existente o ingresar nuevos datos. Sus características principales son:

- Recibe la entrada de datos de la capa de presentación.
- Interactúa con la capa de datos para realizar operaciones.
- Manda los resultados procesados a la capa de presentación.

- Capa de datos:

Aqui residen los datos y es la encargada de acceder a los mismos. Está formada por uno o más gestores de bases de datos que realizan todo el almacenamiento de datos, reciben solicitudes de almacenamiento o recuperación de información desde la capa de negocio. Para ello se utilizan APIs como pueden ser JDBC, de bajo nivel para acceder a las diferentes fuentes de datos.

Sus características principales son:

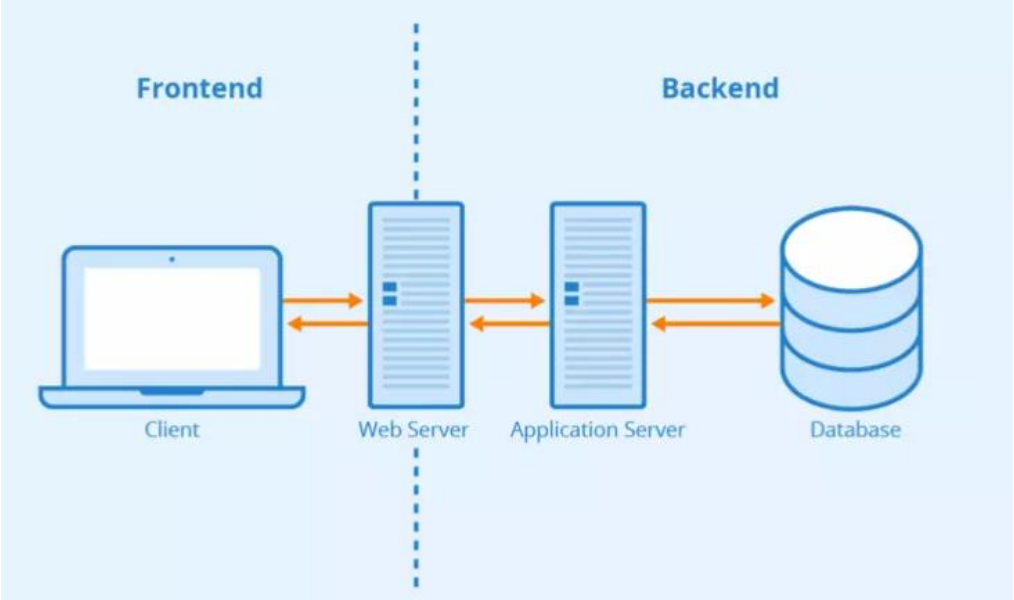
- Almacena los datos.
- Recupera datos.
- Mantiene los datos.
- Asegura la integridad de los datos.

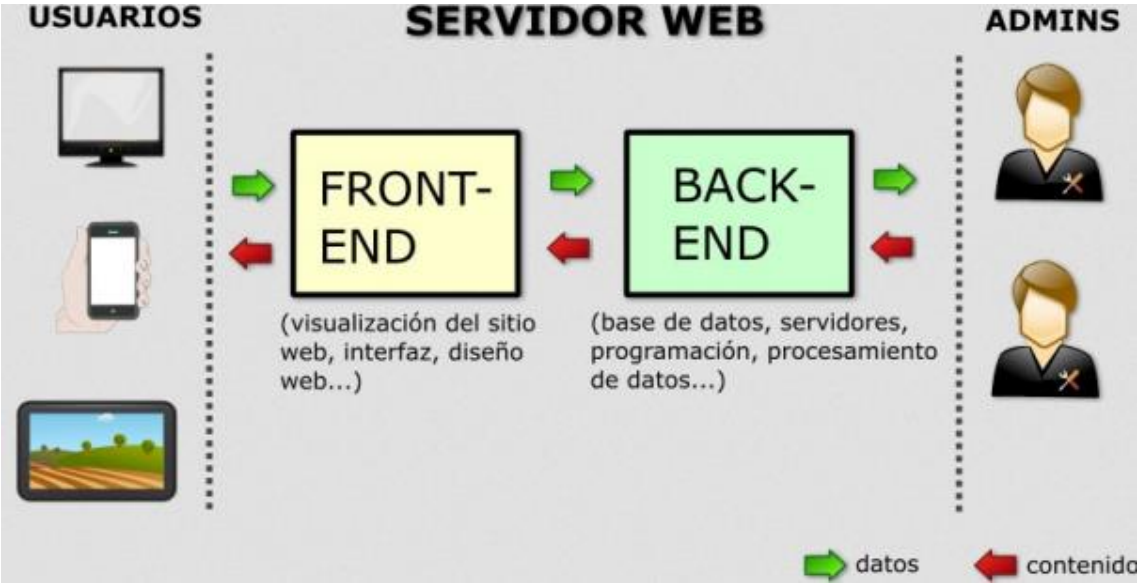


6. Front-end / Back-end

El front-end es la parte visual e interactiva que se ejecuta en el navegador del usuario, enfocada en el diseño y la experiencia. La funcionalidad del back-end gestionando la lógica de negocio, la seguridad y las bases de datos para alimentar de información al front-end. El back-end es accesible para los desarrolladores, pero también es accesible para cualquier otro usuario de la empresa que tenga que gestionar datos. Por ejemplos publicadores y administradores.

Front-end incluye lo que se ejecuta en el navegador del cliente (HTML, CSS, JavaSrcipt). Back-end es el .





Enlace de interés: https://es.wikipedia.org/wiki/Front_end_y_back_end

7. Tipos de web

La evolución pe las páginas web nació con la web estática, la cual muestra contenido fijo en HTML. Mientras que una dinámica genera su contenido en tiempo real desde una base de datos. Con los años se desarrollaron aplicaciones web es un software interactivo en el navegador. Finalmente, un mashup es un sitio que integra datos o APIs de múltiples fuentes externas en una única interfaz.

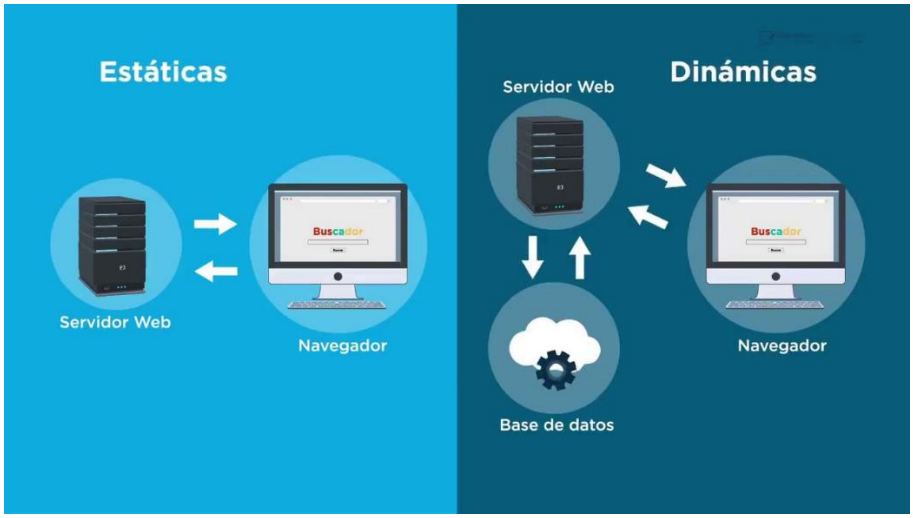
Estática: Es una página web en la cual el contenido no varía. El usuario se limita a obtener dicha información sin poder interactuar con la página web.

Una página web estática está compuesta por archivos HTML individuales por cada página que son pre generados y presentados al usuario a través del navegador de la misma forma.

Dinámica: Pagina web que permite crear aplicaciones dentro de la propia web, dando una mayor interactividad con el cliente.

Una página web dinámica incluye aspectos que se caracterizan por la interactividad y la funcionalidad, por ejemplo, los usuarios pueden interactuar con la información que se presenta en la página gracias a las instrucciones creadas a través de los lenguajes de programación y la base de datos sobre la que está construida.

Los sitios web dinámicos basan su comportamiento y funcionalidad en dos tipos de programación, front-end (del lado del cliente) y back-end (del lado del servidor). Las instrucciones del lado del cliente es código JavaScript que se ejecuta en el navegador. Mientras que las instrucciones que se ejecutan del lado del servidor son instrucciones escritas en lenguajes de scripting o programación, como .Net, PHP, Java… y que son ejecutadas para crear lo que el usuario ha solicitado en su interacción con la página.



Aplicación Web: Conjunto de herramientas que los usuarios pueden utilizar accediendo a un servidor web mediante un navegador, dicha página almacenara la información del usuario en su servidor.

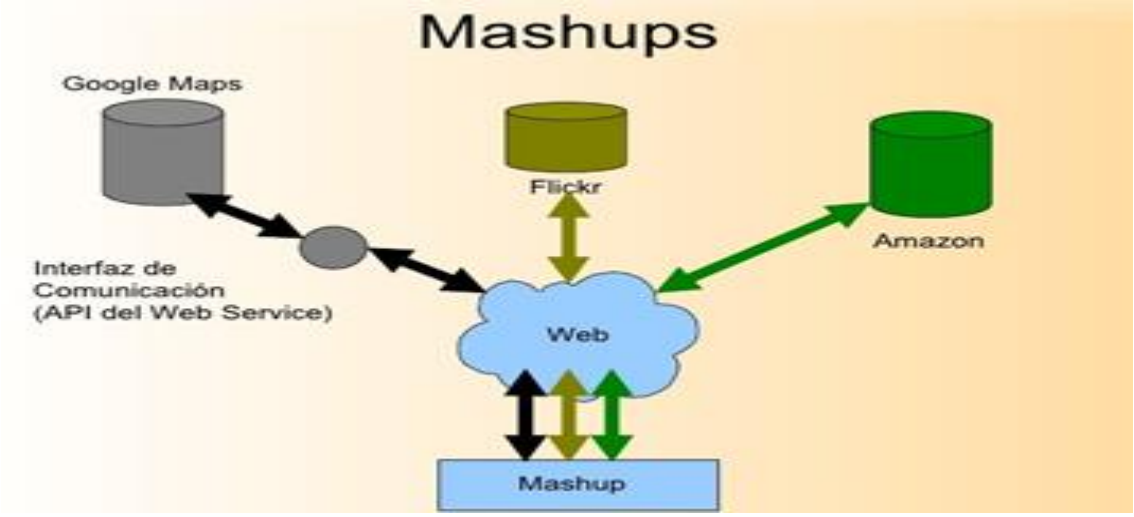
Las aplicaciones web son populares debido a lo práctico del navegador web como cliente ligero, a la independencia del sistema operativo, así como a la facilidad para actualizar y mantener aplicaciones web sin distribuir e instalar software a miles de usuarios potenciales.

Se pueden dividir en tres capas:

- Capa del navegador o presentación
- Capa del servidor o negocio
- Capa de persistencia o de datos.

Mashup: Es una aplicación web híbrida que combina servicios de diversas páginas web en una sola para ofrecer un nuevo servicio, se basan en utilizar contenido de terceros a través de fuentes o interfaces públicas.

Las empresas pueden crear experiencias innovadoras sin que los usuarios tengan que salir de sus páginas web. A su vez, las fuentes de mashup logran una mayor visibilidad que si sus servicios solo se ofrecieran a través de su página web.



Aplicaciones web complejas caracteriasticas

Aplicación web- dinámica control de acceso y base de datos

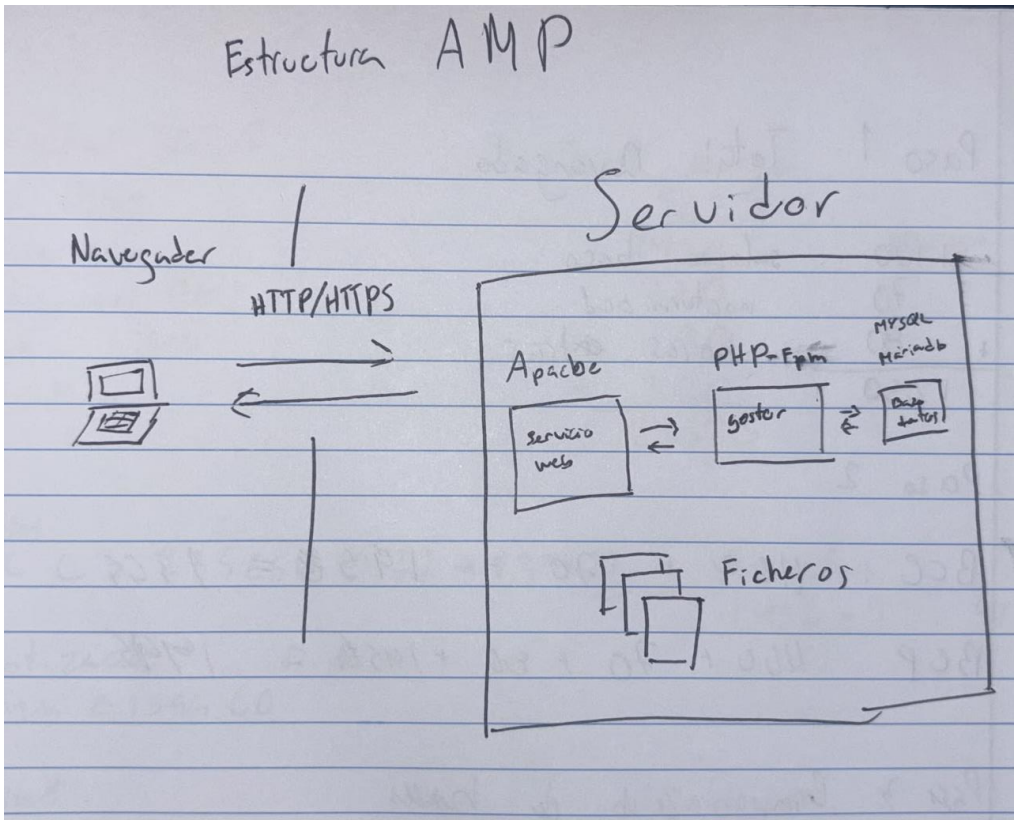
Aplicación web multicapa-

Web service-usar y ofrecer

Aplicación web hibrida -> apliaccoin web compleja mas de una arquitectura diferentes duenos

8. Componentes de una aplicación web

- **Servidor web**: Es el programa informático que recibe las peticiones (solicitudes HTTP/HTTPS) de los navegadores de los usuarios y se encarga de devolver la respuesta correspondiente, ya sea entregando archivos estáticos (imágenes, HTML) o derivando la petición al motor de la aplicación. Ejemplo de arquitectura AMP: Apache.
- **Módulo encargado de ejecutar el código**: Es el entorno de ejecución o motor que procesa la lógica de la aplicación dinámica. Recibe la petición del servidor web, ejecuta las instrucciones necesarias y devuelve el resultado. Ejemplo de arquitectura AMP: PHP-fpm
- **Sistema gestor de base de datos (si es necesario)**: Es el software diseñado para almacenar, gestionar y recuperar la información persistente de la aplicación de manera estructurada y segura (como cuentas de usuario, inventarios o publicaciones). Ejemplo de arquitectura AMP: Mysql/Mariadb
- **(Ficheros escritos en) Lenguajes de programación**: Son los archivos físicos que contienen el código fuente desarrollado por los programadores. Este código dicta las reglas de negocio y cómo debe comportarse la aplicación. Ejemplo de arquitectura AMP: fichreos php, html, sql



Enlace de interés: https://platzi.com/blog/que-son-aplicaciones-web/

9. Programas en cliente y servidor

Los programas del cliente se ejecutan en el navegador utilizando JavaScript, HTML y CSS para manejar la interfaz y la interactividad visual. Los programas del servidor procesan datos y seguridad de forma invisible al usuario utilizando lenguajes como Python, Java, PHP o Ruby.

Cliente: Se utilizan navegadores —Google Chrome, Mozilla Firefox, Safari, etc.—, que se comunican con los servidores e interpretan lenguajes tipados como HTML y CSS, y JavaScript o TypeScript.

Lenguajes y tecnologías del lado de cliente
HTML: Lenguaje de marcas que indica la estructura de los contenidos de una página.
CSS: Da estilo a las páginas añadiendo márgenes, colores o tipografías.
JavaScript: Lenguaje que crea pequeños programas que realizan acciones dentro de una página web. Se insertan en HTML mediante la etiqueta <script>.

Servidor: Ejecuta procesos de negocio y acceso a datos. Lenguajes: PHP, Java, C#, Ruby, Node.js.

Lenguajes y tecnologías del lado del servidor
CGI: El sistema más antiguo para páginas dinámicas. Algo desfasado en la actualidad. Se suele utilizar Perl, aunque también en C, C++ o Visual Basic.
Perl: Lenguaje interpretado y extensible, se conecta muy bien con otros lenguajes.
ASP: Tecnología de Microsoft para páginas dinámicas de servidor. Se escribe en la propia página web con VBScript o JScript (Javascript de Microsoft).
PHP (Hypertext Preprocessor): Lenguaje rápido e independiente de la plataforma con una gran librería de funciones y una extensa documentación.
JSP (JavaServer Pages): Tecnología que permite crear páginas de servidor en lenguaje Java. Es multiplataforma.

Base de datos: Almacenan información necesaria para las aplicaciones. Los sistemas gestores de bases de batos las administran mediante queries. Ejemplos son MariaDB, MySQL o MongoDB, que se diferencia de las anteriores en que su almacenamiento de datos no es relacional.

Categoría	**Descripción**	**Instrucciones**
DDL—Data Definition Language	Especifican el esquema de la base de datos.	-create -alter -drop -rename
DML — Data Manipulation Language	Modifican la información almacenada.	-insert -update -delete -select
DCL —Data Control Language	Controlan los permisos de acceso a los datos.	-grant -revoke
TCL — Transaction Control Language	Llevan a cabo grupos de tareas.	-commit -rollback -savepoint

10. Lenguajes lado servidor

Lenguajes como PHP, Python, Java, C# y JavaScript (Node.js) dominan el back-end por su capacidad para conectar bases de datos y procesar lógica pesada. Actualmente, Node.js lidera en crecimiento por su rendimiento y ecosistemas modernos, mientras que Java y C# mantienen una presencia masiva en entornos corporativos.

PHP: De los más utilizados actualmente, es de código abierto y muy rápido. Es fácil de aprender y multiplataforma. Permite establecer conexión con todo tipo de bases de datos, como Oracle o MySQL. El servidor, no obstante, no delega tareas al lado del cliente. PHP es código embebido, se escribe dentro de etiquetas html. Por ejemplo:

<?php ejemplo ?>

Java / C#: Altamente implantados en entornos corporativos, muy robustos y tipados.

Node.js (JavaScript): Alta adopción moderna para operaciones asíncronas y de tiempo real.

Python: Creciente popularidad gracias a la inteligencia artificial y a frameworks como Django.

Server-side Programming Languages

Most popular server-side programming languages

© W3Techs.com	usage	change since 1 August 2026
1. PHP	69.9%	-0.7%
2. JavaScript	7.5%	+0.6%
3. Ruby	7.1%	+0.2%
4. Java	5.5%	
5. Scala	5.0%	

percentages of sites

Fastest growing server-side programming languages since 1 August 2026

© W3Techs.com	sites
1. JavaScript	105.3
2. Ruby	33.0
3. Scala	0.5

daily increase of number of sites per million

Find more details in the server-side language surveys

Via **w3techs**

11. Plataforma XAMPP

XAMPP es un paquete de software gratuito que instala rápidamente un entorno de desarrollo local que incluye el servidor web Apache, la base de datos MariaDB y los intérpretes de PHP y Perl. Permite a los desarrolladores programar y probar aplicaciones web en sus propios equipos sin necesidad de un hosting web o conexión a internet.

Paquete de software libre que agrupa de forma preconfigurada un servidor web Apache, base de datos MariaDB, y los lenguajes PHP y Perl (en sistemas multiplataforma "X").

Permite crear entornos de desarrollo web locales muy rápidamente.

componentes principales de XAMPP:

Apache: Es el servidor web más popular en internet, Apache es la aplicación de servidor web que procesa y entrega el contenido web al cliente.

MySQL: Es el sistema gestor de base de datos más popular y de codigo abierto. Esta implementado en websites de aficionados hasta plataformas profesionales como WordPress. Cada aplicación web, ya sea simple o compleja, requiere una base de datos para almacenar los datos que recoge.

PHP: Es un lenguaje de script del lado del servidor que hace funcionar algunas de las páginas web más famosas del mundo, como WordPress o Facebook. Es de código abierto, relativamente fácil de aprender, y funciona perfectamente con MySQL, lo que lo ha hecho una opción muy popular para los desarrolladores web.

Perl: Es un lenguaje de programación dinámico y de alto nivel que se usa ampliamente en la programación de redes, la administración de sistemas, etc. Aunque menos popular para el desarrollo web, Perl tiene un montón de aplicaciones específicas.

12. Uso de JVM y JDK

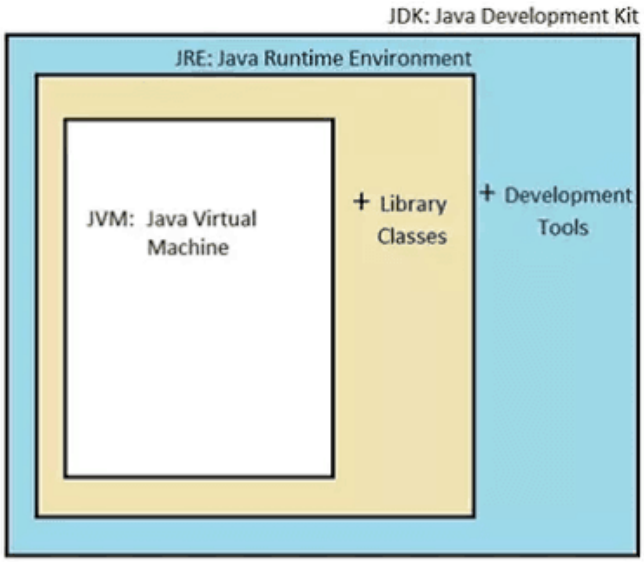
El JDK (Java Development Kit) es indispensable únicamente en el entorno de desarrollo porque contiene el compilador y las herramientas para escribir código. Por otro lado, la JVM (Java Virtual Machine) es estrictamente necesaria tanto en desarrollo como en explotación (servidor), ya que es el motor que ejecuta el código Java ya compilado.

El JDK contiene los compiladores y librerías, siendo estrictamente necesario en el entorno de desarrollo para programar.

La JVM es la máquina que ejecuta el código, siendo obligatoria tanto en el entorno de desarrollo como en el de explotación (servidor en producción).

Enotorno de desarrollo ambas

Entorno de explotación jvm



JDK = JRE + Development Tools
JRE = JVM + Library Classes

13. IDE más utilizados

Un IDE (Entorno de Desarrollo Integrado) centraliza un editor de código, depurador y consola, siendo Visual Studio Code el más utilizado a nivel global por su ligereza y soporte de extensiones. Otros entornos con altísima implantación en la industria incluyen IntelliJ IDEA y Eclipse para el ecosistema Java, y Visual Studio para el entorno .NET.

Visual Studio Code (el más popular y versátil, multilenguaje), IntelliJ IDEA (dominante en desarrollo Java), PhpStorm (líder de pago para PHP) y Eclipse/NetBeans (clásicos, perdiendo algo de mercado).

Todos ellos soportan múltiples lenguajes de programación como Java, PHP, C++, Python..., aunque algunos deben ser descargados vía plugin. También suelen incluir un intérprete o un compilador de código.

14. Servidores HTTP/HTTPS

Nginx y Apache son los servidores web líderes absolutos en el mercado para recibir peticiones y despachar archivos. Nginx destaca por su velocidad y uso eficiente de memoria bajo tráfico extremo, mientras que Apache sigue siendo popular por su flexibilidad y configuración mediante archivos

Nginx (alto rendimiento, servidor de proxy inverso, gran implantación actual), Apache HTTP (muy extendido, uso de módulos .htaccess) e Cloudflare (red de entrega de contenido, servicios de seguridad de Internet y servicios de servidores de nombres de dominio distribuidos).

Web Servers

Most popular web servers

© W3Techs.com	usage	change since 1 August 2026
1. Cloudflare Server	30.9%	+1.9%
2. Nginx	30.9%	-0.6%
3. Apache	22.0%	-1.1%
4. LiteSpeed	14.5%	-0.5%
5. Node.js	7.2%	+0.4%

percentages of sites

Fastest growing web servers since 1 August 2026

© W3Techs.com	sites
1. Cloudflare Server	358.3
2. Node.js	80.5
3. Google Servers	7.7

daily increase of number of sites per million

15. Apache HTTP vs Apache Tomcat

Apache HTTP es un servidor web tradicional de propósito general diseñado para entregar contenido estático y enrutar tráfico. Apache Tomcat, en cambio, es un contenedor especializado diseñado específicamente para compilar y ejecutar aplicaciones web escritas en el ecosistema de Java.

Apache HTTP es un servidor web generalizado de contenidos (estáticos y dinámicos como PHP).

Apache Tomcat es un contenedor de servlets específicamente diseñado para desplegar aplicaciones web desarrolladas en Java (JSP/Servlets).

Algunas diferencias son:

- La diferencia fundamental es que Tomcat proporciona contenido dinámico en Java, mientras que el propósito principal del servidor web Apache es simplemente servir contenido estático como HTML, imágenes, audio y texto.
- Tomcat se creó específicamente para aplicaciones Java, mientras que Apache es un servidor HTTP de propósito general. Puedes utilizar Apache junto con diferentes lenguajes de programación (PHP, Python, Perl, etc.) con la ayuda del módulo de Apache apropiado al lenguaje.
- En cuanto a velocidad, Apache HTTP es más rápido que Apache Tomcat en lo que a las páginas web estáticas se refiere.

Enlaces de Interes:

Apache Tomcat vs Apache HTTP

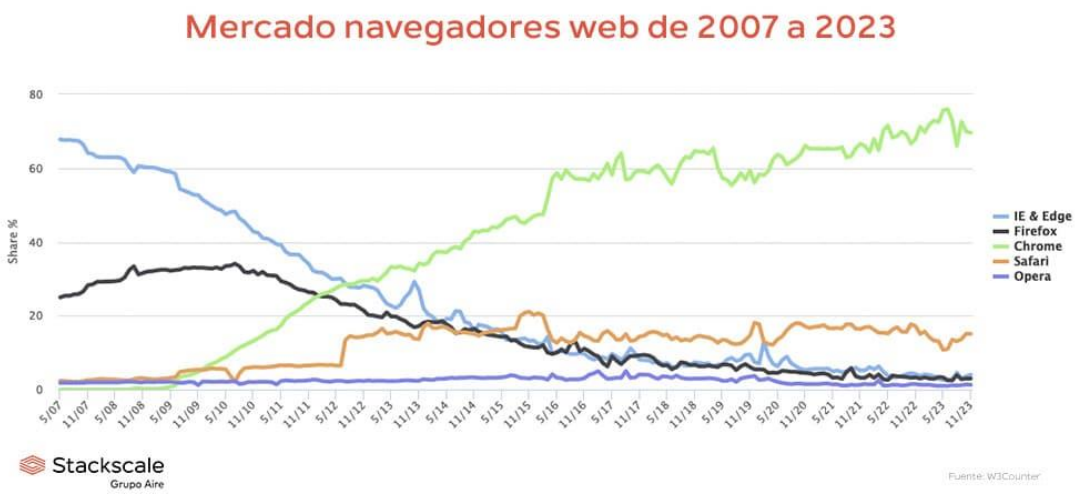
Apache HTTP

Apache Tomcat

16. Navegadores

Google Chrome domina ampliamente el mercado con más del 60% de cuota global, seguido a distancia por Safari (Apple), Edge (Microsoft) y Firefox. Su función principal es actuar como clientes universales que solicitan recursos al servidor y renderizan HTML, CSS y JavaScript de manera visual y segura.

Google Chrome (líder absoluto), Safari (dominante en ecosistema Apple), Microsoft Edge (ganando usuarios corporativos) y Mozilla Firefox (basado en software libre y privacidad).



Via: https://linuxpunx.com.au/top-browsers-2023-comparison-and-ranking-stackscale/

Google Chrome: Rápido, sencillo e intuitivo. Tiene una fuerte seguridad, con actualizaciones automáticas. Como inconveniente consume demasiados recursos del PC.

Safari: Es el navegador que se suele utilizar en los Mac. Su sencillez le da estabilidad y eficiencia, aunque tampoco es el más rápido. La falta de actualizaciones hace que tenga menos seguridad.

Mozilla Firefox: Es uno de los más veteranos. Tiene actualizaciones constantes y su funcionamiento es estable. Es fácil de personalizar con extensiones.

Microsoft Edge: Es la alternativa más modernizada de Microsoft a Internet Explorer. Está más actualizado y es más rápido. Consume pocos recursos del PC.

Opera: También es veterano. Similar a Google Chrome, con un apartado para guardar páginas web. Tan personalizable como Chrome o Firefox.

Internet Explorer: Está actualmente obsoleto y por lo tanto tiene muchos problemas de seguridad. Su velocidad tampoco destaca.

Generación de documentación html?

Javadoc, phpdoc: estándares de etiquetado para generar documentación

@package appFinal @author Sasha @since

Cms sistema gestor de contenidos con ejemplos wordpress organizar recursos en la web

Erp sap Navision empresarial